

Homework 5

张子乐,3230104237

2026-04-25

1. 题目

考虑广义线性混合模型：

$$y_i | \mathbf{b} \sim \text{Poi}(\mu_i)$$

其中 $\log(\mu_i) = \mathbf{X}_i \boldsymbol{\beta} + \mathbf{Z}_i \mathbf{b}$, $\mathbf{b} \sim N(\mathbf{0}, \Lambda^{-1})$.

虽然观测值 $\mathbf{y}$ 与随机效应 $\mathbf{b}$ 的对数联合概率函数 $\log\{f(\mathbf{y}, \mathbf{b})\}$ 容易写出，但进行似然推断需要求出边缘概率函数：

$$f_y(\mathbf{y}) = \int f(\mathbf{y}, \mathbf{b}) d\mathbf{b}$$

它的计算依赖于数值积分.

2. 模拟数据生成与相关函数定义

生成 50 个模拟数据，其中：

- 固定效应部分为 $\beta_0 + \beta_1 x$
- 随机效应部分由因子 z_1 和 z_2 组成，每个因子有 2 个水平
- $\text{diag}(\Lambda) = [\sigma_1^{-2}, \sigma_1^{-2}, \sigma_2^{-2}, \sigma_2^{-2}]$

2.1 生成模拟数据

```
set.seed(0); n <- 50
x <- runif(n)
z1 <- sample(c(0,1), n, replace=TRUE)
z2 <- sample(c(0,1), n, replace=TRUE)
b <- rnorm(4) ## simulated random effects
X <- model.matrix(~x)
```

```

Z <- cbind(z1,1-z1,z2,1-z2)
eta <- X%%c(0,3) + Z%%b
mu <- exp(eta)
y <- rpois(mu,mu)
theta <- c(0,3,1,1)

```

2.2 对数联合概率函数

函数 `lf.yb0` 用于计算 $\log\{f(\mathbf{y}, \mathbf{b})\}$

- `theta` contains the fixed effect parameters, i.e. β and the parameters of Λ .
- `lambda` is the name of a function for turning the parameters of Λ into its leading diagonal elements, for example `var.func`.

函数 `lf.yb` 是 `lf.yb0` 的向量化版本，可以接受一个矩阵 `b`，其中每一列是一个随机效应的取值，从而充分利用 R 的向量化特性加速计算。

```

# evaluate log{f(y,b)}
lf.yb0 <- function(y,b,theta,X,Z,lambda) {
  beta <- theta[1:ncol(X)]
  theta <- theta[-(1:ncol(X))]
  eta <- X%%beta + Z%%b
  mu <- exp(eta)
  lam <- lambda(theta,ncol(Z))
  sum(y*eta - mu - lfactorial(y)) - sum(lam*b^2)/2 + sum(log(lam))/2 - length(lam)/2*log(2*pi)
}
# example of parameter lambda
var.func <- function(theta,nb) c(theta[1],theta[1],theta[2],theta[2])

```

```

# evaluate log{f(y,b)} (can accept a matrix b)
lf.yb <- function(y, b, theta, X, Z, lambda) {
  if (is.null(dim(b))) b <- matrix(b, ncol = 1) # 回退
  beta <- theta[1:ncol(X)]
  theta <- theta[-(1:ncol(X))]
  eta <- as.vector(X %% beta) + Z %% b # n x K
  mu <- exp(eta) # n x K
  lam <- lambda(theta, ncol(Z))
  poisson <- colSums(y * eta - mu - lfactorial(y)) # log P(y/b)
  normal <- - colSums(lam * b^2) / 2 + sum(log(lam)) / 2 - length(lam)/2*log(2*pi) # log P(b)
}

```

```

return(poisson + normal)
}

```

3. Laplace approximation

Algorithm: for $I = \int \exp(\ell(b)) db$

Require: Smooth $\ell(b)$ on $\mathbb{R}^d$.

- 1: Find $\hat{b} \leftarrow \arg \max_b \ell(b)$ (e.g. Newton or quasi-Newton).
- 2: Compute $H \leftarrow -\nabla^2 \ell(\hat{b})$.
- 3: Output $\log \hat{I}_{\text{Lap}} \leftarrow \ell(\hat{b}) + \frac{d}{2} \log(2\pi) - \frac{1}{2} \log |H|$.

3.1 计算梯度和 Hessian 矩阵

对数联合概率函数为:

$$L = \log f(\mathbf{y}, \mathbf{b}) = \sum_i \{y_i(\mathbf{X}_i \beta + \mathbf{Z}_i \mathbf{b}) - \exp(\mathbf{X}_i \beta + \mathbf{Z}_i \mathbf{b})\} - \frac{1}{2} \mathbf{b}^T \Lambda \mathbf{b} + C$$

令 $\mu = \exp(\mathbf{X}\beta + \mathbf{Z}\mathbf{b})$.

- 1. 梯度 $\nabla_{\mathbf{b}} L$:

$$\mathbf{g} = \frac{\partial L}{\partial \mathbf{b}} = \mathbf{Z}^T (\mathbf{y} - \mu) - \Lambda \mathbf{b}$$

- 2. Hessian 矩阵 $\nabla_{\mathbf{b}}^2 L$:

$$\mathbf{H} = \frac{\partial^2 L}{\partial \mathbf{b} \partial \mathbf{b}^T} = -\mathbf{Z}^T \text{diag}(\mu) \mathbf{Z} - \Lambda$$

```

# 梯度
glf.yb <- function(y, b, theta, X, Z, lambda) {
  beta <- theta[1:ncol(X)]
  theta <- theta[-(1:ncol(X))]
  eta <- as.vector(X %*% beta) + as.vector(Z %*% b)
  mu <- exp(eta)
  lam <- lambda(theta, ncol(Z))
  g <- t(Z) %*% (y - mu) - lam * b
  return(as.vector(g))
}

```

```

# Hessian
hlf.yb <- function(y, b, theta, X, Z, lambda) {
  beta <- theta[1:ncol(X)]
  theta <- theta[-(1:ncol(X))]
  eta <- as.vector(X %*% beta) + as.vector(Z %*% b)
  mu <- exp(eta)
  lam <- lambda(theta, ncol(Z))
  H <- -t(Z) %*% (as.vector(mu) * Z) - diag(lam)
  return(H)
}

```

3.2 求解极大值点并计算 Laplace approximation

```

time_laplace <- system.time({
  b <- rep(0,4)
  while(TRUE) {
    b0 <- b
    g <- glf.yb(y,b,theta,X,Z,var.func)
    H <- hlf.yb(y,b,theta,X,Z,var.func)
    b <- b - solve(H,g) # 牛顿法
    if (sum(abs(b-b0))<1e-10*sum(abs(b))) break
  }
  laplace_result <- as.numeric(
    lf.yb(y,b,theta,X,Z,var.func) + 2*log(2*pi) -0.5*as.numeric(determinant(H,logarithm=TRUE)$modu
  )
})

```

```

cat(" 极大值点 b =", round(b,4), "\n")
极大值点 b = 0.5675 -0.4887 0.4684 -0.3896
cat("Laplace approximation 估计:", laplace_result, "\n")
Laplace approximation 估计: -119.2717
cat(" 耗时:", time_laplace["elapsed"], " 秒\n")
耗时: 0.02 秒

```

4. Laplace-IS

Laplace-IS 的核心思想是取 $g(b) = \mathcal{N}(\hat{b}, H^{-1})$ 为建议分布, 其中 $\hat{b} \leftarrow \arg \max_b \ell(b)$, $H \leftarrow -\nabla^2 \ell(\hat{b})$.

```

seeds <- c(0, 42, 123, 2026, 9999, 4237, 66, 888, 20220602, 1) # 不同的随机数种子
N <- 10000
R <- chol(-H)

time_lis <- system.time({
  laplace_is <- sapply(seeds, function(s) {
    set.seed(s)
    z <- matrix(rnorm(N * 4), 4, N)
    bm <- b + backsolve(R, z)
    lf <- lf.yb(y, bm, theta, X, Z, var.func)
    zz.2 <- colSums(z * z) / 2
    lfz <- lf + zz.2
    log(sum(exp(lfz - max(lfz)))) + max(lfz) + 2 * log(2 * pi) - sum(log(diag(R))) - log(N)
  })
})
df_lis <- data.frame(Seed = seeds, `Results` = laplace_is, check.names = FALSE)
knitr::kable(df_lis, digits = 6, caption = "Laplace-IS")

```

表 1: Laplace-IS

Seed	Results
0	-119.2692
42	-119.2710
123	-119.2697
2026	-119.2714
9999	-119.2706
4237	-119.2678
66	-119.2692
888	-119.2707
20220602	-119.2692
1	-119.2692

```

cat("Laplace-IS 均值:", mean(laplace_is), "\n")
Laplace-IS 均值: -119.2698
cat("Laplace-IS 方差:", var(laplace_is), "\n")
Laplace-IS 方差: 1.214289e-06
cat("Laplace-IS 平均耗时:", time_lis["elapsed"]/10, " 秒/次 \n")
Laplace-IS 平均耗时: 0.021 秒/次

```

5. Monte Carlo

使用最简单的平均值法，在以原点为中心的超立方体 $[-m.r/2, m.r/2]^4$ 上均匀采样，其中 $m.r$ 通过前面计算的 $\hat{b}$ 和 H 来确定。

```
seeds <- c(0, 42, 123, 2026, 9999, 4237, 66, 888, 20220602, 1)
Sigma <- solve(-H)
sigma <- sqrt(diag(Sigma))
m.r <- 2 * max(abs(b) + 3 * sigma)
N <- 100000

time_mc <- system.time({
  mc <- sapply(seeds, function(s) {
    set.seed(s)
    bm <- matrix((runif(N * 4) - .5) * m.r, 4, N) + b
    lf <- lf.yb(y, bm, theta, X, Z, var.func)
    log(sum(exp(lf - max(lf)))) + max(lf) - log(N) + log(m.r^4)
  })
})

df_mc <- data.frame(Seed = seeds, `Results` = mc, check.names = FALSE)
knitr::kable(df_mc, digits = 6, caption = "Monte Carlo")
```

表 2: Monte Carlo

Seed	Results
0	-119.1651
42	-119.2428
123	-119.1776
2026	-119.5566
9999	-119.0297
4237	-119.2853
66	-119.6316
888	-119.5809
20220602	-119.3186
1	-119.9040

```
cat("Monte Carlo 均值:", mean(mc), "\n")
Monte Carlo 均值: -119.3892
```

```
cat("Monte Carlo 方差:", var(mc), "\n")
Monte Carlo 方差: 0.07221881
cat("Monte Carlo 平均耗时:", time_mc["elapsed"] / 10, " 秒/次 \n")
Monte Carlo 平均耗时: 0.395 秒/次
```

6. 高斯积分 (Gauss-Hermite Quadrature)

基于 Hermite 正交多项式的根, 获取对应的节点 x_i 和权重 w_i 。

$$\int_{-\infty}^{\infty} f(x)e^{-x^2}dx \approx \sum_{i=1}^k w_i f(x_i)$$

```
time_ghq <- system.time({
  k <- 32 # 节点数
  rule <- gaussHermiteData(k) # 权函数  $e^{-x^2}$ 
  z_nodes <- sqrt(2) * rule$x # 转换为标准正态分布的节点
  w_nodes <- rule$w / sqrt(pi)
  grid <- expand.grid(b1 = z_nodes, b2 = z_nodes, b3 = z_nodes, b4 = z_nodes)
  weight_grid <- expand.grid(w1 = w_nodes, w2 = w_nodes, w3 = w_nodes, w4 = w_nodes)
  weights <- weight_grid$w1 * weight_grid$w2 * weight_grid$w3 * weight_grid$w4
  bm_ghq <- t(as.matrix(grid))
  # 剔除正态先验部分
  log_likelihood_posion <- function(y, b, theta, X, Z) {
    beta <- theta[1:ncol(X)]
    eta <- as.vector(X %*% beta) + Z %*% b
    mu <- exp(eta)
    colSums(y * eta - mu - lfactorial(y))
  }
  vals <- log_likelihood_posion(y, bm_ghq, theta, X, Z)
  log_w <- log(weights)
  llw <- log_w + vals
  max_llw <- max(llw)
  ghq_result <- log(sum(exp(llw - max_llw))) + max_llw
})
cat("Gauss-Hermite Quadrature 估计:", ghq_result, "\n")
Gauss-Hermite Quadrature 估计: -119.2602
cat(" 耗时:", time_ghq["elapsed"], " 秒\n")
耗时: 2.46 秒
```

7. 总结

表 3: 四种方法估计 $\log f_y(\mathbf{y})$ 的对比

方法	估计值	耗时 (秒)
Laplace Approximation	-119.2717	0.020
Laplace-IS (均值)	-119.2698	0.021
Monte Carlo (均值)	-119.3892	0.395
Gauss-Hermite Quadrature	-119.2602	2.460

- 1. 对比两种具有随机性的方法, Laplace-IS 的方差 1.2142886×10^{-6} 明显小于 Monte Carlo (平均值法) 的方差 0.0722188。其根据极大值点和 Hessian 矩阵构造建议分布, 利用了目标函数峰值的信息。
- 2. Monte Carlo (平均值法) 的耗时较长。其在超立方体上均匀采样, 在高维空间中效率极低。
- 3. 最初尝试取 8 个高斯积分节点时, Gauss-Hermite Quadrature 的耗时较短, 但其估计值与其他方法存在较大差异, 可能是因为标准高斯积分的节点是根据 $N(0, 1)$ 选择的, 偏离了目标函数的峰值区域, 且节点数过少本身就会产生较大的误差。增加节点数到 32 后, 估计值与其他方法接近, 但耗时显著增加。
- 4. 文献中指出, 将估计量的 “design points” 集中在目标函数具有较大数值的区域, 可能能够使方差最小化。上面的分析印证了这点。